

Course code : **CSE3009**
Course title : **No SQL Data Bases**
Module : **4**
Topic : **4**

Replication AND Sharding

Objectives

This session will give the knowledge about

- Replication
- Sharding

Replication

A replica set is **a group of MongoDB instances that store the same set of data.**

Replicas are basically used in production to ensure a high availability of data.

Redundancy and data availability:

Because of replication, we have redundant data across the MongoDB instances.

We are **using replication to provide a high availability** of data to the application.

If one instance of MongoDB is unavailable, we can serve data from another instance.

Replication

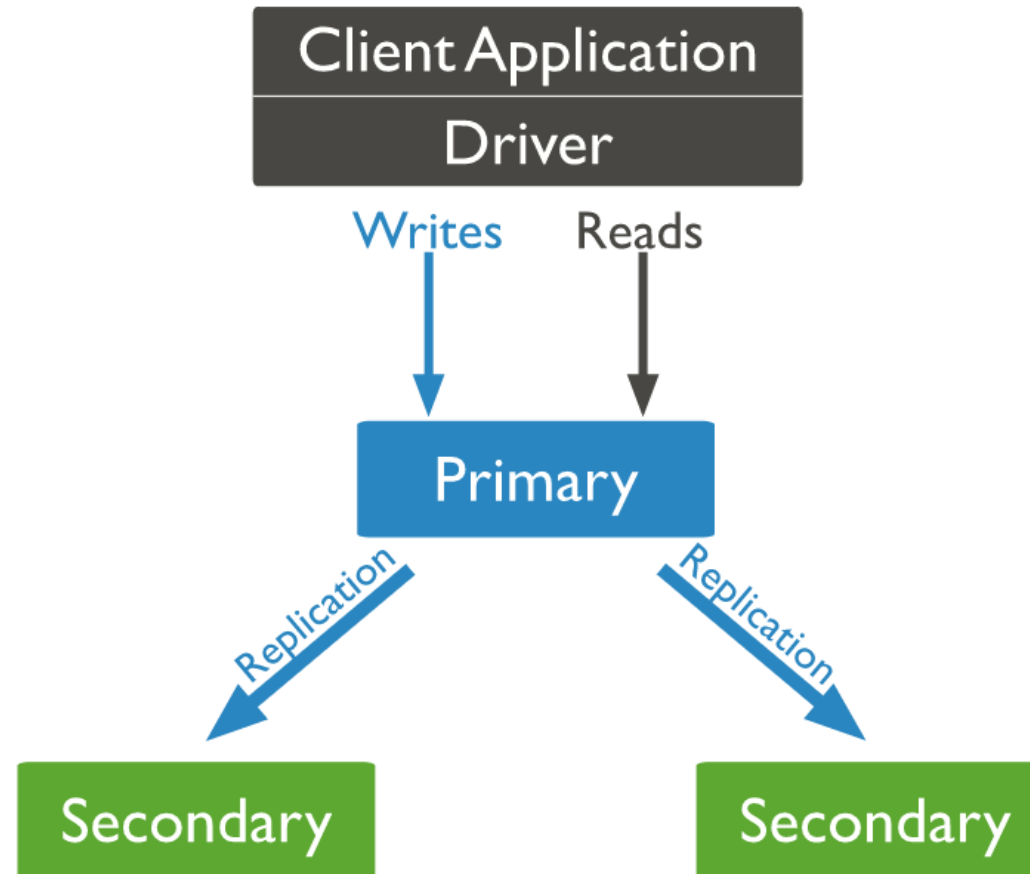
Replication also **increases the read capacity of applications** as reading operations can be sent to different servers and retrieve data faster.

By maintaining data on different servers, we can increase the locality of data and increase the availability of data for distributed applications.

We can use the replica copy for backup, reporting, as well as disaster recovery.

The primary node receives all write operations. A replica set can have only one primary capable of confirming writes with { w: "majority" } write concern; although in some circumstances, another mongod instance may transiently believe itself to also be primary.

Replication



Working with replica sets

A replica set is a group of MongoDB instances that have the same dataset. **A replica set has one arbiter node and multiple data-bearing nodes.**

In data-bearing nodes, one node is considered the primary node while the other nodes are considered the secondary nodes.

All write operations happen at the primary node.

Once a write occurs at the primary node, the data is replicated across the secondary nodes internally to make copies of the data available to all nodes and to avoid data inconsistency.

Working with replica sets

If a primary node is not available for the operation, secondary nodes use election algorithms to select one of their nodes as a primary node.

A special node, called an **arbiter node**, is added in the replica set. **This arbiter node does not store any data.**

The arbiter is used to maintain a quorum in the replica set by responding to a heartbeat and election request sent by the secondary nodes in replica sets.

As an arbiter does not store data, it is a cost-effective resource used in the election process.

Working with replica sets

If votes in the election process are even, the arbiter adds a voice to choose a primary node.

The arbiter node is always the arbiter, it will not change its behavior, unlike a primary or secondary node.

The primary node can step down and work as secondary node, while secondary nodes can be elected to perform as primary nodes.

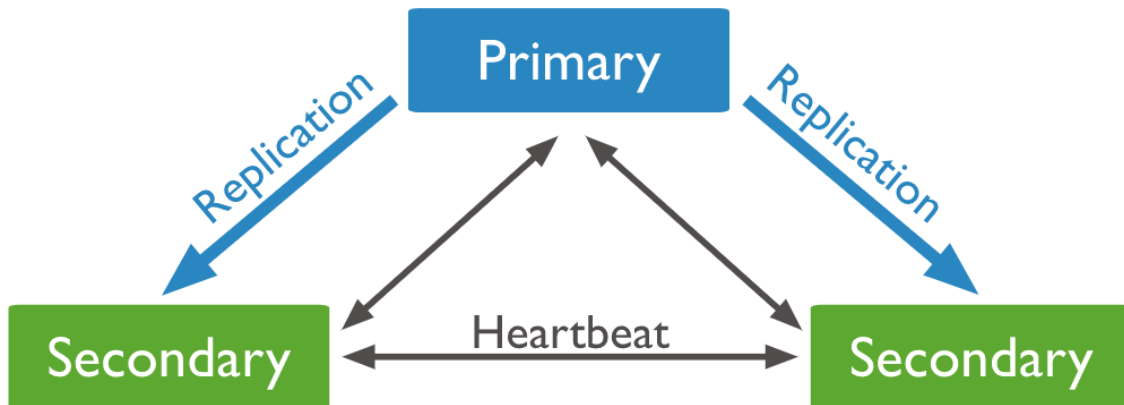
Secondary nodes apply read/write operations from a primary node to secondary nodes asynchronously.

Replication

The secondaries replicate the primary's oplog and apply the operations to their data sets such that the secondaries' data sets reflect the primary's data set.

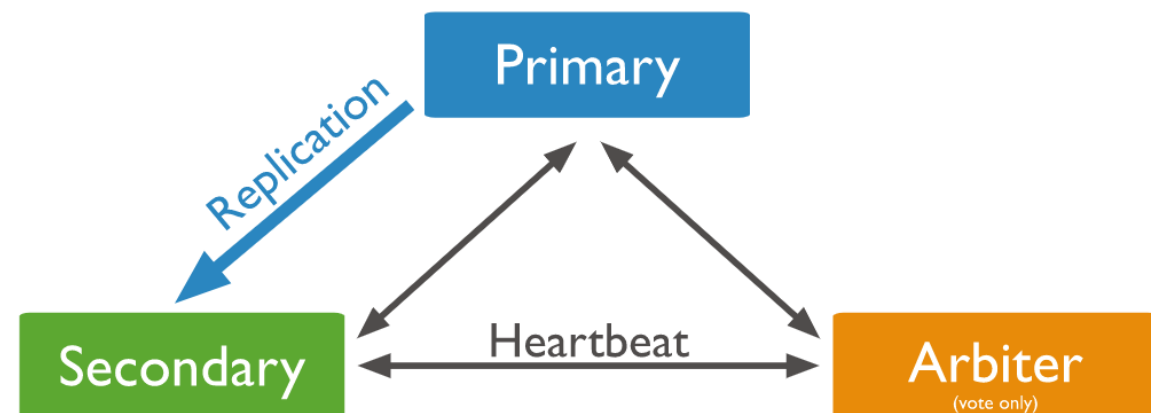
If the primary is unavailable, an eligible secondary will hold an election to elect itself the new primary.

For more information on secondary members, see Replica Set Secondary Members.



Replication

In some circumstances (such as you have a primary and a secondary but cost constraints prohibit adding another secondary), you may choose to add a mongod instance to a replica set as an arbiter.



An arbiter participates in elections but does not hold data (i.e. does not provide data redundancy). For more information on arbiters, see [Replica Set Arbiter](#).

Automatic failover in replication

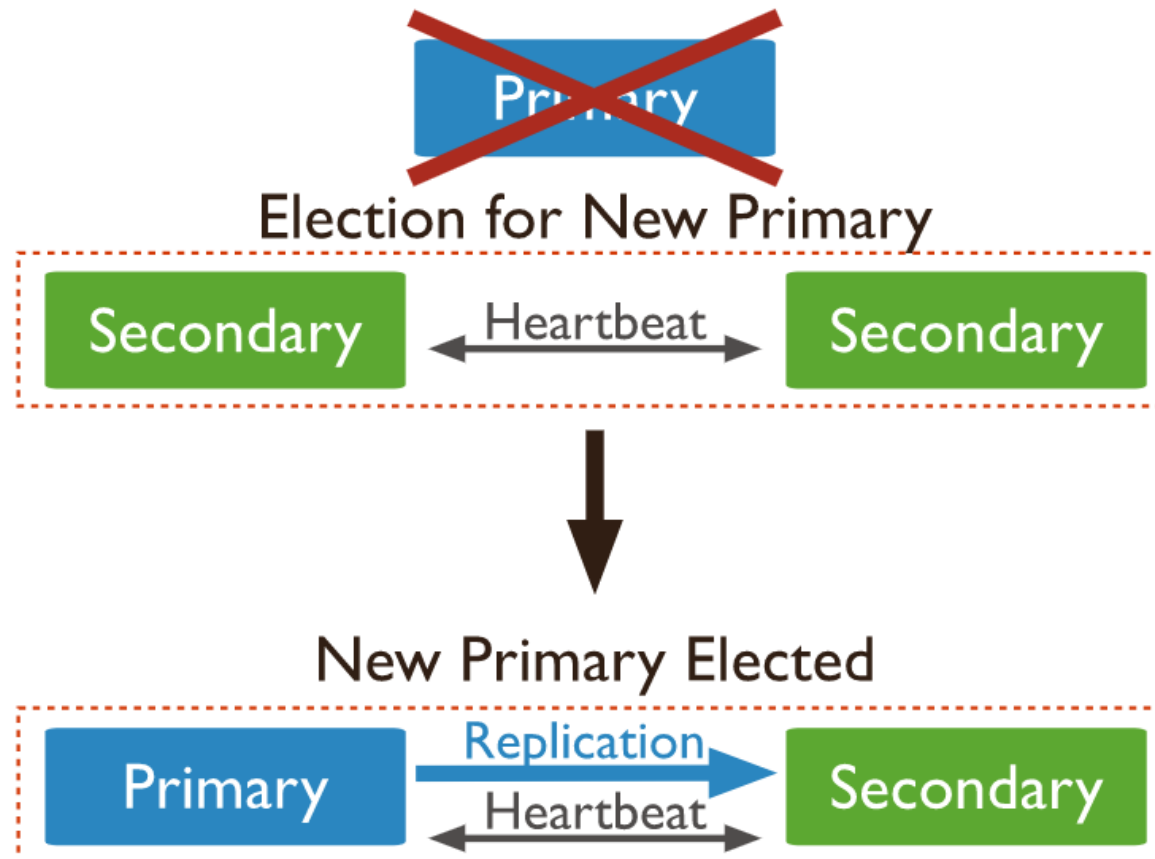
Primary nodes always communicate with **other members every 10 seconds**.

If it fails to communicate with the others in 10 seconds, other eligible secondary nodes hold an election to choose a primary-acting node among them.

The **first secondary node that holds the election and receives the majority of votes is elected as a primary node**.

If there is an arbiter node, its vote is taken into consideration while choosing primary nodes.

Automatic failover in replication

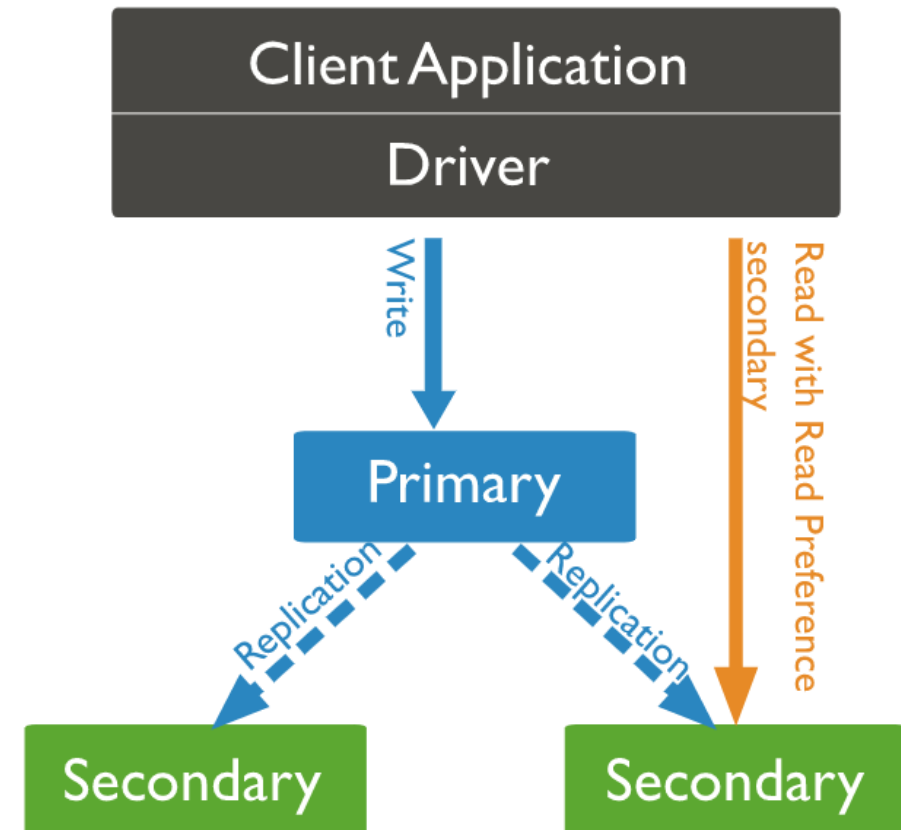


Read operations

Basically, the read operation happens at the primary node only, but we can specify the read operation to be carried out from secondary nodes also.

A read from a secondary node does not affect data at the primary node.

Reading from secondary nodes can also give inconsistent data.



Sharding

Sharding is a methodology to **distribute data across multiple machines**.

Sharding is basically used for deployment with a large dataset and high throughput operations.

The single database cannot handle a database with large datasets as it requires larger storage, and bulk query operations can use most of the CPU cycles, which slows down processing.

For such scenarios, we need more powerful systems.

Sharding

One approach is to add more capacity to a single server, such as adding more memory and processing units or **adding more RAM on the single server**, this is also called **vertical scaling**.

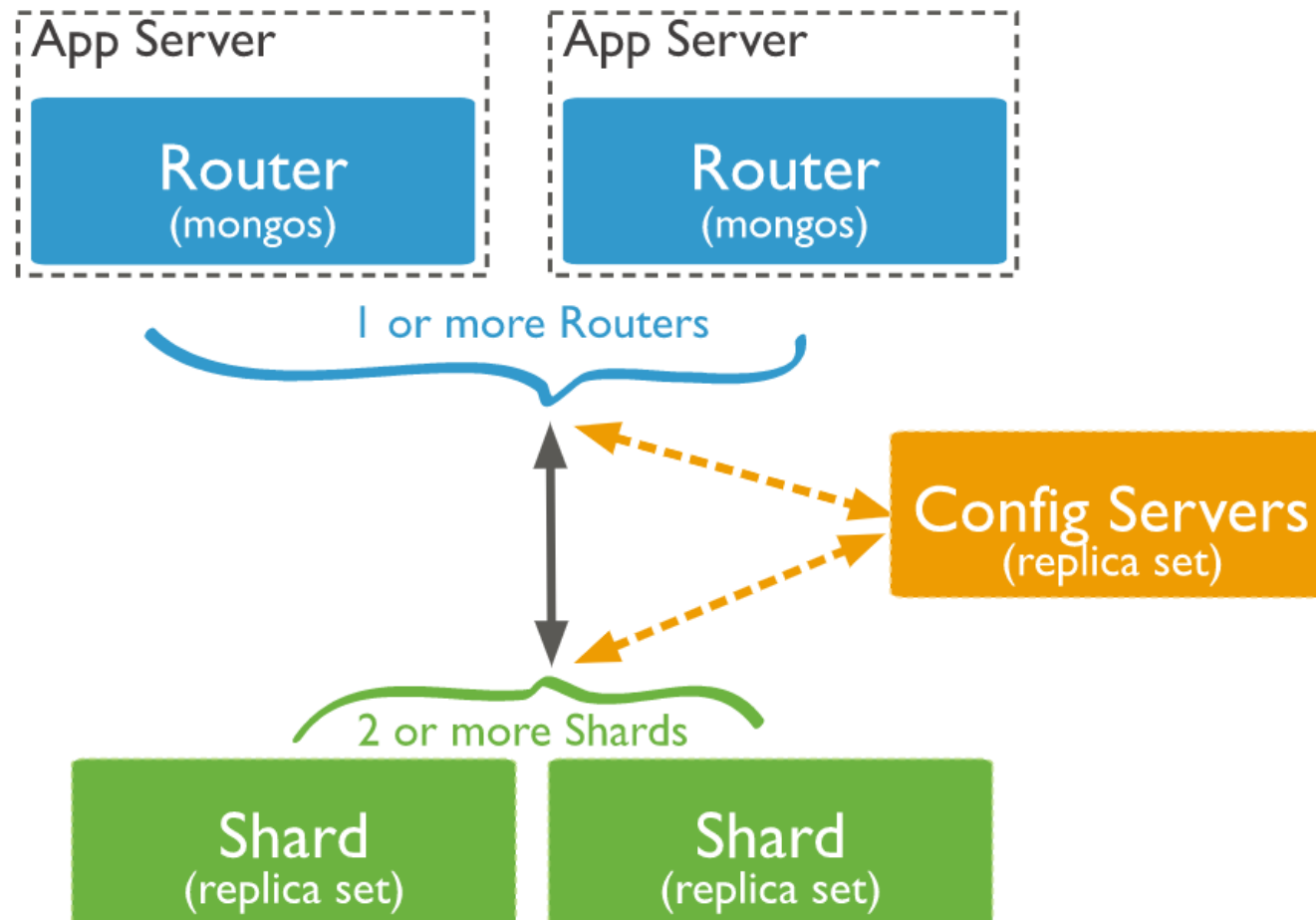
Another approach is to **divide a large dataset across multiple systems** and serve a data application to query data from multiple servers. This approach is called **horizontal scaling**.

MongoDB handles horizontal scaling through sharding.

Sharded clusters: Components

- **Shard:** Each shard stores a subset of sharded data. Also, each shard can be deployed as a replica set.
- **Mongos:** Mongos provide an interface between a client application and sharded cluster to route the query.
- **Config server:** The configuration server stores the metadata and configuration settings for the cluster. The MongoDB data is sharded at the collection level and distributed across sharded clusters.
- **Shard keys:** To distribute documents in collections, MongoDB partitions the collection using the shard key. MongoDB shards data into chunks. These chunks are distributed across shards in sharded clusters.

Sharded clusters: Components



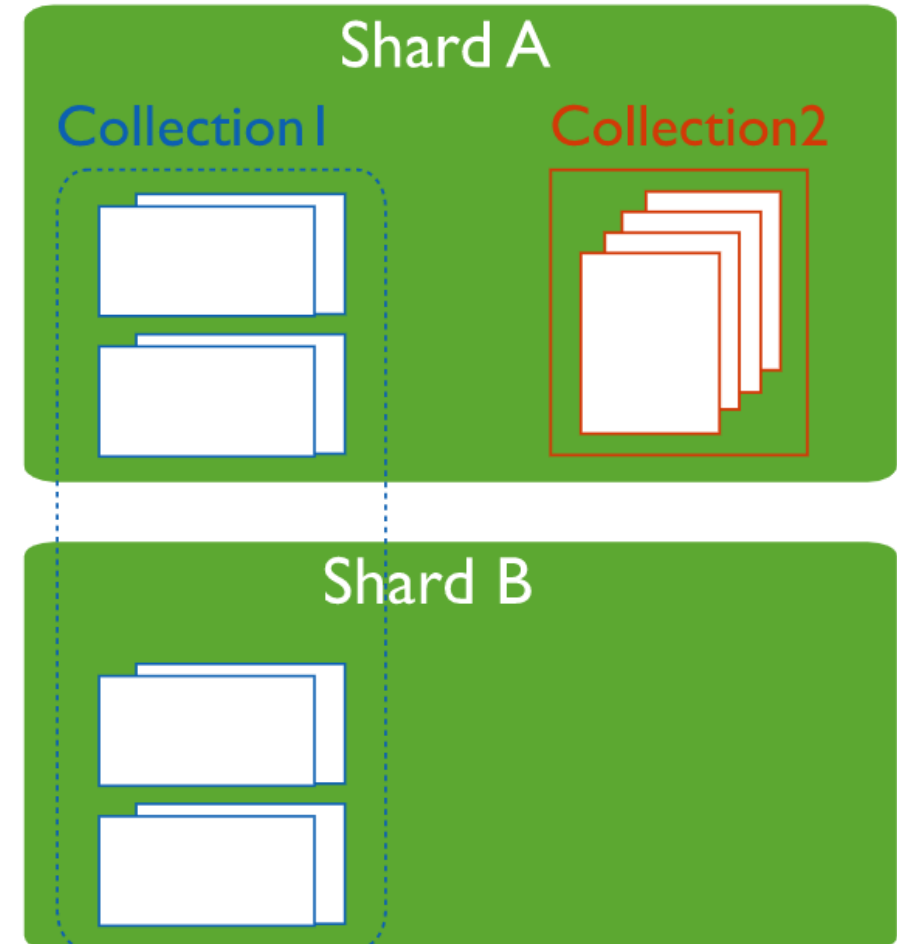
Advantages of sharding

- When we use sharding, the load of the read/write operations gets distributed across sharded clusters.
- As sharding is used to distribute data across a shard cluster, we can increase the storage capacity by adding shards horizontally.
- MongoDB allows continuing the read/write operation even if one of the shards is unavailable.
- In the production environment, shards should deploy with a replication mechanism to maintain high availability and add fault tolerance in a system.

Sharded and Non-Sharded Collections

A database can have a mixture of sharded and unsharded collections. Sharded collections are partitioned and distributed across the shards in the cluster.

Unsharded collections are stored on a primary shard. Each database has its own primary shard.

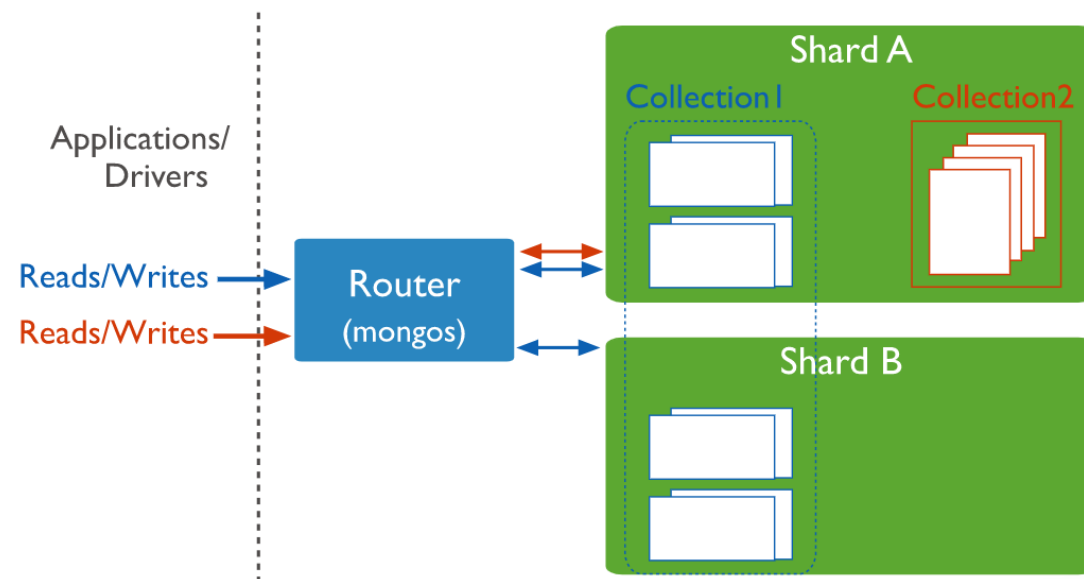


Connecting to a Sharded Cluster

You must connect to a mongos router to interact with any collection in the sharded cluster.

This includes sharded and unsharded collections.

Clients should never connect to a single shard in order to perform read or write operations.



Sharding Strategy

MongoDB supports two sharding strategies for distributing data across sharded clusters.

Hashed Sharding

Hashed Sharding involves computing a hash of the shard key field's value. Each chunk is then assigned a range based on the hashed shard key values.

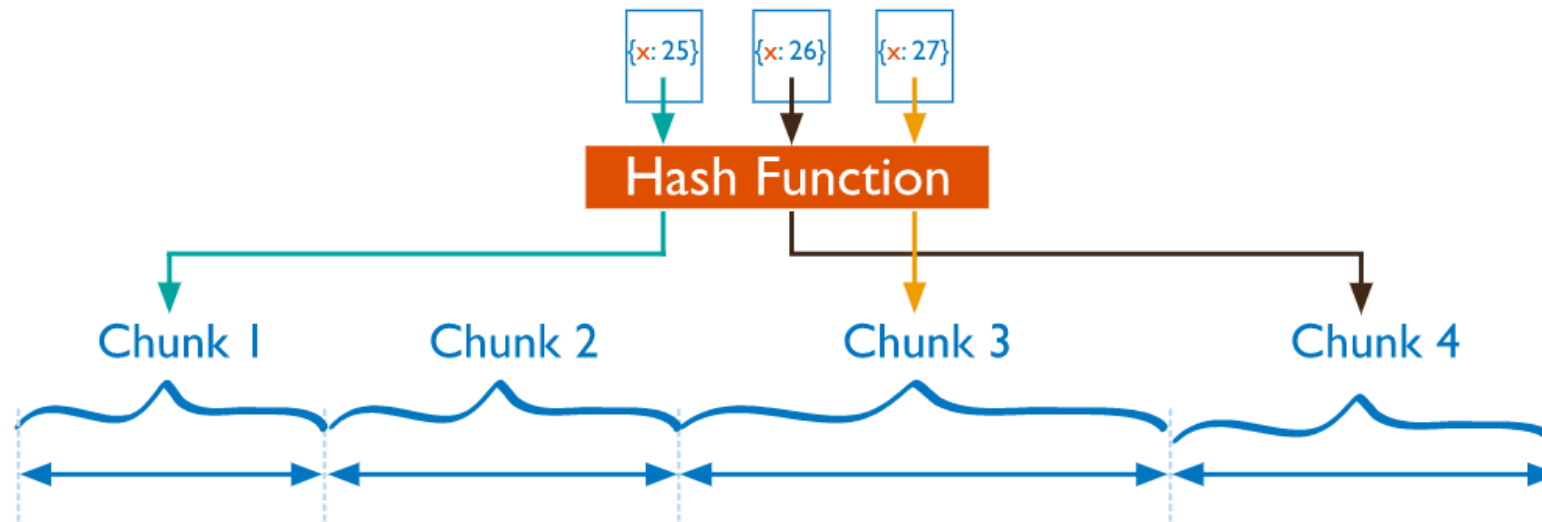
MongoDB automatically computes the hashes when resolving queries using hashed indexes. Applications do not need to compute hashes.

While a range of shard keys may be “close”, their hashed values are unlikely to be on the same chunk.

Sharding Strategy

Data distribution based on hashed values facilitates more even data distribution, especially in data sets where the shard key changes monotonically.

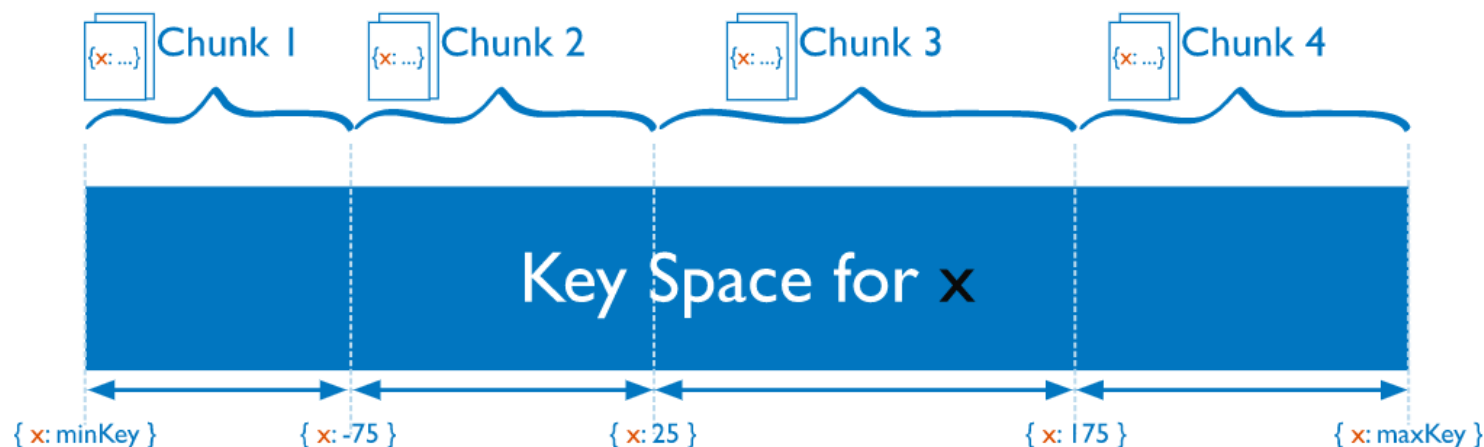
However, hashed distribution means that range-based queries on the shard key are less likely to target a single shard, resulting in more cluster wide broadcast operations



Ranged Sharding

Ranged sharding involves dividing data into ranges based on the shard key values. Each chunk is then assigned a range based on the shard key values.

A range of shard keys whose values are “close” are more likely to reside on the same chunk. This allows for targeted operations as a mongos can route the operations to only the shards that contain the required data.



Storing large data in MongoDB

- MongoDB is document-based database, data is stored in JSON and XML documents.
- MongoDB has a **document size limit of 16 MB**. If the size of a JSON document exceeds 16 MB, instead of storing data as a single file, **MongoDB divides the file into chunks** and each chunk is stored as a document in the system.
- MongoDB creates a **chunk of 255 KB** to divide files and only the last chunk can have less than 255 KB.

Storing large data in MongoDB

- MongoDB uses two collections to work with gridfs.
- One collection is used to store the chunk data and another collection is used to store the metadata.
- When you query MongoDB for the operation of the gridfs file, MongoDB uses the metadata collection to perform the query and collect data from different chunks. GridFS stores data in two collections:
 - chunks: Stores binary chunks.
 - files:: Stores the file's metadata.

Summary

This session will give the knowledge about

- Replication
- Sharding
- References:
 - <https://docs.mongodb.com/manual/tutorial/>
 - “Seven NoSQL Databases in a Week” Author: Aaron Ploetz Aaron